

Dynamic Graph Neural Networks

Executive summary

Dynamic (time-varying) GNNs extend static message passing to settings where **edges, nodes, and features evolve over time**. The field has converged on three dominant input regimes: **snapshot sequences** (discrete-time graphs), **continuous-time event streams** (timestamped interactions), and **streaming/online updates** (graphs too large or too fast to store/process in full). Surveys emphasize that many apparent “model improvements” in temporal link prediction can be confounded by **evaluation leakage and easy negative sampling**, and that simple memorization baselines can be surprisingly strong when edges repeat.

1

The most influential architectural patterns are: (i) **GNN + sequence model** (RNN/GRU/LSTM) over snapshots or over node embeddings; (ii) **temporal attention** with explicit time encoding; (iii) **memory-based continuous-time frameworks** that update node states per event; and (iv) **temporal point-process models** where event intensity depends on learned embeddings.

2

Benchmarks have matured: **TGB (NeurIPS 2023)** provides large-scale datasets and standardized, realistic protocols for dynamic link prediction and dynamic node property prediction; **DGB (NeurIPS 2022)** focuses on tightening evaluation for dynamic link prediction and highlights edge recurrence and negative-sampling pitfalls.

3

Open research gaps that intersect strongly with spectral and multiscale/perceptual ideas include: **spectral stability under topology change**, **frequency-domain characterization of temporal shifts**, and **multiscale temporal coarsening** (learning evolving “cells”/communities that become the units of reasoning). Recent papers already take steps in this direction via evolving Fourier transforms for temporal graphs and spectral-domain shift handling.

4

Taxonomy and mathematical formalisms

Dynamic GNNs can be organized along three orthogonal axes: temporal granularity, update mechanism, and what is allowed to change.

Temporal granularity

Snapshot-based (discrete time): The data is a sequence of graph snapshots $G_1, G_2, \dots, G_T$. Models typically run a static GNN per snapshot and then connect snapshots with an RNN or temporal attention. EvolveGCN is a canonical snapshot model that evolves **GCN parameters** across time using an RNN, avoiding reliance on fixed node embeddings and explicitly targeting changing node sets.

5

Continuous-time / event-based: The data is a sequence of timestamped interactions $(u, v, t, \mathbf{x}_{uv}(t))$. Models update states at event times and often sample temporal neighborhoods (“most recent k”). TGAT and

TGN are canonical: TGAT uses temporal attention with functional time encoding; TGN is a general framework combining node memories with message passing on event streams. ⁶

Streaming/online: The graph arrives as a stream of edges (and possibly attributes) with strict constraints on storing history and retraining. “Streaming Graph Neural Networks” (SIGIR 2020) explicitly formulates coherent online updates with time intervals and propagation over edge streams. Systems work (TGL, DistTGL, GNNFlow) focuses on training at massive scale and/or distributed settings, often for memory-based temporal GNNs. ⁷

Update mechanisms

Recurrent updates: Temporal dynamics appear via an RNN on node embeddings or on GNN parameters. EvolveGCN evolves the GCN weights with GRU/LSTM variants. ⁵

Temporal attention: Time is treated as an input to attention over temporal neighbors. TGAT introduces a functional time encoding grounded in Bochner’s theorem and uses self-attention to aggregate temporal-topological neighborhoods. ⁸

Memory modules + event message passing: TGN defines per-node memory states, a message function combining source/destination memory with time encoding and raw event features, an aggregator, and a memory updater; it can cast multiple prior methods as instances. ⁹

Temporal point processes (TPP): Events are generated/assessed via intensities $\lambda(t)$ conditioned on embedding history. DyRep models dynamic graphs as interleaving processes (e.g., communication events and association/topology evolution) mediated by latent embeddings, and parameterizes conditional intensity functions with neural networks. ¹⁰

ODE/SDE formulations: Continuous-depth or continuous-time evolution is modeled via differential equations. Graph Neural ODEs (Graph Neural Differential Equations / GDEs) define continuous-depth GNN dynamics; CT-GNs (ODE-based temporal graph networks) model continuous embedding evolution over dynamic graph data. ¹¹

Modeling choices: what changes over time

Birth/death and churn: Nodes/edges may appear/disappear; this is central in interaction graphs and streaming settings. TGN’s event-based view naturally handles edge arrivals and supports inductive vs transductive tasks; EvolveGCN explicitly motivates changing node sets across snapshots. ¹²

Attribute dynamics vs topology rewiring: Some domains assume fixed topology with evolving node signals (spatiotemporal graphs), while others emphasize evolving connectivity. DCRNN and Graph WaveNet focus on time series over a graph (road networks), with mechanisms to capture temporal dynamics and (in Graph WaveNet) an adaptive learned dependency matrix. ¹³

Spectral-temporal operators: A new line explicitly models changing spectra or frequency components. EFT proposes an “evolving graph Fourier transform,” and SILD/FreeDyG emphasize frequency-domain patterns and distribution shifts on dynamic graphs. ⁴

Suggested mermaid taxonomy diagram (for slides/notes):

```

flowchart TD
    A[Dynamic Graph Learning] --> B[Snapshot-based]
    A --> C[Continuous-time / Event-based]
    A --> D[Streaming / Online]

    B --> B1[GNN per snapshot + RNN/Attention]
    B --> B2[Parameter evolution (EvolveGCN)]

    C --> C1[Temporal attention (TGAT)]
    C --> C2[Memory + message passing (TGN)]
    C --> C3[TPP intensity models (DyRep)]
    C --> C4[ODE-based embedding dynamics (CT-GN/CTAN)]

    D --> D1[Edge-stream updates (StreamingGNN)]
    D --> D2[Systems: billion-scale training (TGL/DistTGL/GNNFlow)]

```

Representative architectures and key papers

The table below highlights representative models across regimes. Complexity is stated qualitatively because exact cost depends on sampling strategy, depth, and batching.

Model	Input regime	Temporal mechanism	Handles churn	Typical tasks	Notes / complexity cues
EvolveGCN ⁵	snapshots G_t	RNN evolves GCN parameters	yes (motivated)	node/edge/link prediction	avoids fixed node embeddings; cost $\approx$ sum over snapshots of GCN cost + RNN over weights
DySAT ¹⁴	snapshots	structural self-attn + temporal self-attn	partially	node/link tasks	attention over neighbors per snapshot + attention over time
TGAT ⁸	event stream	temporal attention + functional time encoding	yes	link prediction, node classification	time encoding based on harmonic analysis; neighbor sampling critical

Model	Input regime	Temporal mechanism	Handles churn	Typical tasks	Notes / complexity cues
TGN ¹⁵	event stream	node memory + message passing	yes	future edge prediction, dyn node classification	memory update per event; often sample most recent k neighbors; stores per-node memory
DyRep (ICLR'19) ¹⁶	continuous-time events (often)	temporal point process intensities	yes	link/event prediction	explicit conditional intensity; couples topology & interaction dynamics
JODIE (KDD'19) ¹⁷	interaction sequences	coupled RNN updates + future projection operator	yes (interaction setting)	next interaction, state change	introduces time-consistent batching (t-Batch) for scalability
StreamingGNN (SIGIR'20) ¹⁸	edge streams	online recurrent update + propagation	yes	node classification, link tasks	designed for streaming updates with time intervals; maintains evolving node info
CT-GNs (ODE-based) ¹⁹	continuous-time graphs	neural ODE for node dynamics	yes	event-based tasks	models embeddings continuously; dynamic models as discretizations
CTAN (ICML'24) ²⁰	continuous-time events	anti-symmetric ODE for long-range propagation	yes	long-range synthetic + TGB tasks	targets long-range spatio-temporal propagation; discusses dynamic over-squashing limits

Model	Input regime	Temporal mechanism	Handles churn	Typical tasks	Notes / complexity cues
Graph WaveNet (IJCAI'19) ²¹	time series on graph	dilated temporal conv + (learned) adaptive adjacency	topology usually fixed + adaptive	traffic forecasting	handles long temporal horizons via dilated conv; learns hidden spatial dependency

A few “what to remember” mechanisms that recur:

Memory-based event GNN core loop: maintain $\mathbf{m}_v$ for each node v ; for an event $(u, v, t, \mathbf{x})$, compute time encoding $\phi(\Delta t)$, form message $msg(u, v, t)$, aggregate messages per node, update memory (e.g., GRU), and compute embeddings for prediction. TGN explicitly defines this modular decomposition and reports that sampling the **most recent** temporal neighbors can be better than uniform sampling in its ablations. ²²

Evaluation settings matter: In TGN’s experiments on Wikipedia/Reddit/Twitter, they use chronological splits (70/15/15) and report transductive and inductive future-edge prediction; the datasets have timestamped interactions and (for Wikipedia/Reddit) dynamic labels (e.g., whether a user is banned). ²³

Datasets, benchmarks, and evaluation protocols

Temporal interaction networks and dynamic link prediction

Canonical small/medium benchmarks used by early event-based models include Wikipedia and Reddit interaction graphs (bipartite), and a Twitter retweet graph derived from RecSys Challenge data; TGN documents these datasets, feature types, and chronological splits. ²³

Evaluation pitfalls are now well-documented. DGB (“Towards Better Evaluation for Dynamic Link Prediction,” NeurIPS 2022 D&B) shows that edges often reoccur, proposes the **EdgeBank** memorization baseline, and introduces more challenging negative sampling strategies to better match real-world scenarios. ²⁴

TGB (NeurIPS 2023 D&B) is the current community-standard benchmark suite: it provides large-scale datasets across domains (social, trade, transaction, transportation, etc.), includes dynamic link prediction and dynamic node property prediction tasks, and standardizes evaluation pipelines and leaderboards. It reports that model performance varies drastically across datasets and that simple methods can outperform temporal models on some dynamic node property tasks, emphasizing the need for careful baselines. ²⁵

Spatiotemporal forecasting on graphs

Traffic forecasting is the best-known spatiotemporal graph benchmark family. DCRNN models traffic flow as diffusion on a directed graph and combines diffusion convolution with seq2seq recurrent modeling (scheduled sampling), evaluated on large road-network datasets. ²⁶

Graph WaveNet targets long temporal dependencies using stacked dilated 1D temporal convolutions and learns an adaptive dependency matrix via node embeddings to capture latent spatial dependencies. ²¹

Dynamic graphs in vision/sensors

ST-GCN is a canonical model for skeleton-based action recognition, where the “graph” is a human skeleton and the signals change over time; it learns spatial-temporal patterns jointly. ²⁷

Protocol choices you should report explicitly

Temporal split: always split by time (train earlier, test later). TGN uses chronological 70/15/15 splits for its datasets. ²³

Transductive vs inductive: transductive keeps the same node set; inductive tests on new nodes or new node–node pairs. TGN and TGAT explicitly evaluate both (and TGN tabulates both settings). ²⁸

Prediction horizons: for forecasting tasks (traffic), report horizons (e.g., 15/30/60 minutes ahead) and avoid training with information from future time steps (commonly enforced by time-windowed inputs). DCRNN explicitly frames long-term forecasting challenges and uses an encoder–decoder setup for multi-step prediction. ²⁶

OOD / time-shift tests: beyond a single chronological split, assess robustness to seasonalities and structural shifts; spectral/frequency-domain shift papers argue that some shifts are visible in frequency patterns even if not obvious in time-domain statistics. ²⁹

Challenges, failure modes, and best practices

Stability and long-range dependency

Long-range dependency is a known limitation in static GNNs (over-squashing) and carries over—often amplified—into spatiotemporal and continuous-time settings. Alon & Yahav formalize the bottleneck/over-squashing mechanism in GNNs. ³⁰ Recent work explicitly analyzes over-squashing in spatiotemporal GNNs, noting that spatial and temporal receptive fields can compound. ³¹ For continuous-time dynamic graphs, CTAN reports that many message passing, recurrent, and attention methods perform poorly on long-range tasks and proposes an ODE-based anti-symmetric architecture to preserve long-range information flow. ²⁰

Practical best practice: report performance as a function of temporal neighbor sampling depth and model depth; TGN’s ablations show major sensitivity to whether memory is enabled and to the temporal neighbor sampling policy (e.g., “most recent”). ³²

Catastrophic forgetting and continual learning

When training incrementally over time, models can forget previously learned patterns. DyGRAIN (IJCAI 2022) frames an incremental-learning challenge for dynamic graphs as **time-varying receptive fields** and proposes mechanisms to preserve knowledge of informative nodes prone to being forgotten. ³³

Best practice: separate “online inference” (updating embeddings/memory at test time) from “online training” (updating parameters). For continual training, use replay/regularization and report forgetting/forward-transfer metrics (many continual-learning papers exist, but DyGRAIN provides a clear dynamic-graph-specific framing). ³³

Scalability and systems constraints

Memory-based temporal GNNs can be difficult to scale because per-node memories must be maintained consistently. TGL proposes a general framework for temporal GNN training on billion-scale graphs with explicit components (temporal sampler, mailbox, node memory, etc.) and reports large speedups and training feasibility at billion-edge scale. ³⁴ DistTGL targets distributed training of memory-based TGNNs, highlighting synchronization overhead and proposing solutions with near-linear speedups. ³⁵ GNNFlow is a distributed framework for continuous temporal GNN learning on dynamic graphs for multi-GPU machines. ³⁶

Best practice: when reporting throughput, specify whether memory is synchronized, whether neighbor sampling is temporal (CSR-like), and whether training is single-machine vs distributed.

Evaluation leakage and negative sampling

This is one of the most common “silent failure modes.” DGB shows that edge recurrence makes memorization strong and that easy negative sampling can overstate model ability; it introduces EdgeBank and harder negative sampling schemes. ²⁴ TGB reinforces that simple baselines can sometimes outperform complex temporal models, especially in node property prediction, so baseline selection is not optional. ³⁷

Best practices checklist: - ensure negatives are sampled conditionally on time and do not include future information; - report an EdgeBank-like baseline for link prediction when edges repeat; ³⁸ - for inductive tests, explicitly remove new nodes from training history and do not allow them in neighbor caches.

Linking formalisms to spectral geometry and multiscale reasoning

Dynamic GNNs are increasingly incorporating **frequency/spectral views** of temporal evolution, which creates natural bridges to spectral-geometry interests.

EFT (ICLR 2024) proposes an “Evolving Graph Fourier Transform” to address inadequacy and cost of capturing evolving graph spectra, framing it as optimization over Laplacians of continuous-time dynamic graphs and proposing relaxations for efficiency. ³⁹

SILD (NeurIPS 2023) argues that dynamic graph distribution shifts can be “spectral-domain observable” even when time-domain shifts are not, and proposes invariant/variant spectral pattern disentanglement and filtering for improved generalization under distribution shift. ⁴⁰

FreeDyG (ICLR 2024) emphasizes mining frequency-domain interaction patterns (periodic and “shift” behaviors) for continuous-time dynamic link prediction. ⁴¹

These papers suggest a concrete future direction: **temporal PE / spectral diagnostics that remain stable under topology rewiring**, plus **multiscale temporal coarsening** where “perceptual cells” evolve and carry low-frequency structure forward.

A clean intersection with perception-then-reasoning: - Learn evolving mesoscopic “cells” (communities/bottlenecks) as slow variables. - Use event-level updates to refine within-cell dynamics and cross-cell interaction rates. - Enforce spectral stability: cell-graph Laplacian spectrum changes slowly, even if micro-edges churn rapidly.

The motivation is consistent with empirical evidence that long-range propagation can fail in continuous-time dynamic graphs and that specialized architectures (ODE-based, non-dissipative) can help. ⁴²

Minimal pseudocode snippets

Minimal continuous-time event-based GNN loop (TGN-style)

```
# Inputs: event stream E = [(u, v, t, msg)], sorted by time
# State: node memory m[node], last_update[node]
# Modules: time_encode(dt), message_fn(m_u, m_v, msg, time_enc),
#           aggregate(messages_to_node), memory_update(m_node, agg_msg),
#           embed_fn(m_node), predictor(h_u, h_v)

for batch in temporal_batches(E):                # e.g., t-Batch or fixed time
    window
    # 1) Gather and encode events
    msgs_by_node = defaultdict(list)
    for (u, v, t, raw_msg) in batch:
        dt_u = t - last_update[u]
        dt_v = t - last_update[v]
        te_u = time_encode(dt_u)
        te_v = time_encode(dt_v)

        msg_uv = message_fn(m[u], m[v], raw_msg, te_u, te_v)
        msgs_by_node[u].append(msg_uv)
        msgs_by_node[v].append(msg_uv)           # if symmetric; otherwise
    directional

    # 2) Aggregate and update memory
    for node, msg_list in msgs_by_node.items():
        agg = aggregate(msg_list)
        m[node] = memory_update(m[node], agg)
        last_update[node] = batch_end_time(batch)

    # 3) Compute embeddings and loss (e.g., link prediction)
    loss = 0
    for (u, v, t, raw_msg) in batch:
```

```

h_u = embed_fn(m[u])
h_v = embed_fn(m[v])
loss += link_loss(predictor(h_u, h_v), label=1)

# negative sampling at time t (must avoid leakage)
v_neg = sample_negative(u, t)
h_neg = embed_fn(m[v_neg])
loss += link_loss(predictor(h_u, h_neg), label=0)

loss.backward()
optimizer.step()
optimizer.zero_grad()

```

Snapshot-based recurrent GNN training loop (EvolveGCN-style)

```

# Inputs: snapshots G_t = (A_t, X_t, Y_t) for t=1..T
# State: GCN weight parameters W_t that evolve over time
# Modules: gcn_forward(A_t, X_t, W_t), rnn_update(W_t, summary_t), loss_fn

W = init_gcn_weights()
rnn_state = init_rnn_state()

for epoch in range(num_epochs):
    total_loss = 0
    for t in range(1, T_train + 1):
        A_t, X_t, Y_t = G[t]

        # 1) Run GCN with current weights
        H_t = gcn_forward(A_t, X_t, W)

        # 2) Compute task loss at this snapshot
        total_loss += loss_fn(H_t, Y_t)

        # 3) Evolve weights for next snapshot (using an RNN over weights or
        summaries)
        summary_t = summarize_graph(H_t, A_t)      # e.g., pooled node states
        W, rnn_state = rnn_update(W, summary_t, rnn_state)

    total_loss.backward()  # BPTT through time
    optimizer.step()
    optimizer.zero_grad()

```

Recommended reading list and prioritized references

If you read only 10–12 papers first, these give maximal coverage of taxonomy, benchmarks, and core mechanisms:

- Longa et al., “Graph Neural Networks for temporal graphs: State of the art, open challenges, and opportunities” (survey + taxonomy + open challenges). ⁴³
- Kazemi et al., “Representation Learning for Dynamic Graphs: A Survey” (JMLR 2020; encoder–decoder view, datasets). ⁴⁴
- Gravina & Bacciu, “Deep learning for dynamic graphs: models and benchmarks” (rigorous comparisons + baselines). ⁴⁵
- Rossi et al., “Temporal Graph Networks (TGN)” (memory-based event framework; transductive/ inductive; datasets/protocols). ¹⁵
- Xu et al., “TGAT” (temporal attention + functional time encoding). ⁸
- Trivedi et al., “DyRep” (temporal point process + dynamic representation learning; ICLR’19). ¹⁶
- Kumar et al., “JODIE” (coupled RNN updates + future projection; scalable batching). ¹⁷
- Pareja et al., “EvolveGCN” (snapshot-based; evolve GCN parameters). ⁵
- Huang et al., “Temporal Graph Benchmark (TGB)” (NeurIPS 2023; datasets + standardized evaluation). ³⁷
- Poursafaei et al., “Towards Better Evaluation for Dynamic Link Prediction” (NeurIPS 2022; EdgeBank; harder negatives; leakage). ²⁴
- Ma et al., “Streaming Graph Neural Networks” (SIGIR 2020; online edge-stream updates). ¹⁸
- Bastos et al., “Evolving Fourier Transform for Temporal Graphs (EFT)” **or** Zhang et al., “Spectral Invariant Learning...” (spectral-domain shift / dynamic spectra—strong bridge to spectral geometry). ⁴⁶

Additional highly relevant references (especially for scale and long-range dynamics): - TGL (VLDB 2022) and DistTGL (SC 2023) for billion-scale and distributed temporal GNN training. ⁴⁷

- CTAN (ICML 2024) and Over-squashing in STGNNs (2025) for long-range temporal propagation failure modes. ⁴²

- FreeDyG (ICLR 2024) for frequency-enhanced continuous-time link prediction. ⁴⁸

Suggested mermaid update-flow diagram for an event-memory model (for slides):

```
sequenceDiagram
    participant E as Event (u,v,t,msg)
    participant TE as TimeEncoder
    participant MF as MessageFn
    participant AG as Aggregator
    participant MU as MemoryUpdater
    participant EM as EmbedFn
    participant PR as Predictor

    E->>TE: dt_u, dt_v
    TE-->>MF: time encodings
    E->>MF: m_u, m_v, raw msg
```

MF-->>AG: per-node messages
AG-->>MU: aggregated msg per node
MU-->>EM: updated memories
EM-->>PR: h_u , h_v
PR-->>PR: link / node prediction & loss

1 [2302.01018] Graph Neural Networks for temporal graphs: State of the art, open challenges, and opportunities

<https://arxiv.org/abs/2302.01018>

2 5 12 EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs

https://arxiv.org/abs/1902.10191?utm_source=chatgpt.com

3 25 37 Temporal Graph Benchmark for Machine Learning on Temporal Graphs

https://arxiv.org/abs/2307.01026?utm_source=chatgpt.com

4 39 46 Beyond Spatio-Temporal Representations: Evolving Fourier Transform for Temporal Graphs

https://arxiv.org/abs/2402.16078?utm_source=chatgpt.com

6 8 Inductive Representation Learning on Temporal Graphs

https://arxiv.org/abs/2002.07962?utm_source=chatgpt.com

7 18 [1810.10627] Streaming Graph Neural Networks - arXiv.org

https://arxiv.org/abs/1810.10627?utm_source=chatgpt.com

9 15 Temporal Graph Networks for Deep Learning on Dynamic Graphs

https://arxiv.org/abs/2006.10637?utm_source=chatgpt.com

10 16 [1803.04051] Representation Learning over Dynamic Graphs

https://arxiv.org/abs/1803.04051?utm_source=chatgpt.com

11 Graph Neural Ordinary Differential Equations - arXiv.org

https://arxiv.org/pdf/1911.07532?utm_source=chatgpt.com

13 26 Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting

https://arxiv.org/abs/1707.01926?utm_source=chatgpt.com

14 Dynamic Graph Representation Learning via Self-Attention Networks

https://arxiv.org/abs/1812.09430?utm_source=chatgpt.com

17 Predicting Dynamic Embedding Trajectory in Temporal Interaction Networks

https://arxiv.org/abs/1908.01207?utm_source=chatgpt.com

19 arXiv:2205.15924v1 [cs.LG] 31 May 2022

https://arxiv.org/pdf/2205.15924?utm_source=chatgpt.com

20 42 Long Range Propagation on Continuous-Time Dynamic Graphs

<https://arxiv.org/html/2406.02740v1>

21 Graph WaveNet for Deep Spatial-Temporal Graph Modeling

https://arxiv.org/abs/1906.00121?utm_source=chatgpt.com

22 23 28 32 arxiv.org

<https://arxiv.org/pdf/2006.10637>

24 38 Towards Better Evaluation for Dynamic Link Prediction

https://arxiv.org/abs/2207.10128?utm_source=chatgpt.com

27 Spatial Temporal Graph Convolutional Networks for Skeleton-Based Action Recognition

https://arxiv.org/abs/1801.07455?utm_source=chatgpt.com

29 40 Spectral Invariant Learning for Dynamic Graphs under Distribution Shifts

https://arxiv.org/abs/2403.05026?utm_source=chatgpt.com

30 On the Bottleneck of Graph Neural Networks and its Practical Implications

https://arxiv.org/abs/2006.05205?utm_source=chatgpt.com

31 Over-squashing in Spatiotemporal Graph Neural Networks

https://arxiv.org/pdf/2506.15507?utm_source=chatgpt.com

33 DyGRAIN: An Incremental Learning Framework for Dynamic Graphs - IJCAI

https://www.ijcai.org/proceedings/2022/0438.pdf?utm_source=chatgpt.com

34 47 TGL: A General Framework for Temporal GNN Training on Billion-Scale Graphs

https://arxiv.org/abs/2203.14883?utm_source=chatgpt.com

35 DistTGL: Distributed Memory-Based Temporal Graph Neural Network Training

https://arxiv.org/abs/2307.07649?utm_source=chatgpt.com

36 GNNFlow: A Distributed Framework for Continuous Temporal GNN Learning ...

https://arxiv.org/abs/2311.17410?utm_source=chatgpt.com

41 48 FreeDyG: Frequency Enhanced Continuous-Time Dynamic Graph Model for ...

https://proceedings.iclr.cc/paper_files/paper/2024/hash/21f7b745f73ce0d1f9bcea7f40b1388e-Abstract-Conference.html?utm_source=chatgpt.com

43 Graph Neural Networks for temporal graphs: State of the art, open ...

https://arxiv.org/abs/2302.01018?utm_source=chatgpt.com

44 Representation Learning for Dynamic Graphs: A Survey

https://jmlr.org/papers/volume21/19-447/19-447.pdf?utm_source=chatgpt.com

45 Deep learning for dynamic graphs: models and benchmarks

https://arxiv.org/abs/2307.06104?utm_source=chatgpt.com